JAVA INHERITANCE USING INTERFACES

Experiment: Multiple and Multilevel Inheritance Through Interfaces

Objective

To implement inheritance in Java using interfaces and demonstrate how one interface can extend multiple interfaces. The program also shows how a class implements the child interface and provides implementations for all inherited methods.

Task Description

Implement all required inheritance concepts using suitable Java programs. In this experiment, interfaces I2 and I3 are extended by interface I1. Class C1 implements I1 and therefore must implement methods declared in I1, I2, and I3. Execute the program and verify the output using multiple test cases.

Inheritance Demonstrated

• **Multiple inheritance through interfaces:** I1 extends both I2 and I3.

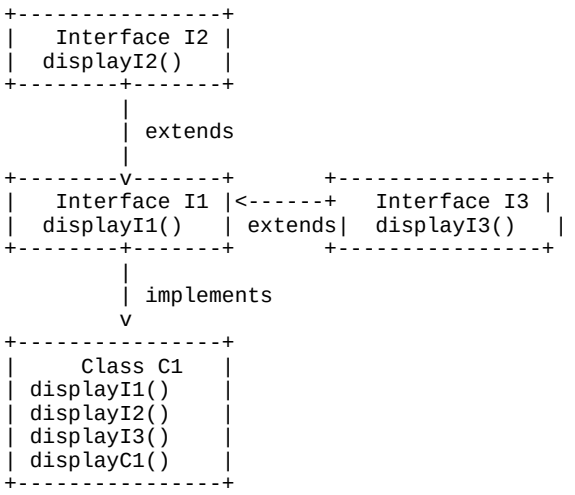
• **Multilevel interface inheritance:** I2/I3 → I1 → C1, where C1 implements the child interface I1.

• **Interface implementation:** C1 implements all abstract methods inherited from I1, I2, and I3.

Algorithm

1. Declare interface I2 with displayI2().
2. Declare interface I3 with displayI3().
3. Declare interface I1 extending both I2 and I3, and declare displayI1().
4. Create class C1 implementing I1.
5. Implement displayI2(), displayI3(), and displayI1() in C1.
6. Create an object of C1 in main().
7. Call all inherited/interface methods and the class method.
8. Observe and verify the output.

UML / Inheritance Structure



Program Code

```
interface I2 {

    // Method declared in Interface I2
    void displayI2();
}

interface I3 {

    // Method declared in Interface I3
    void displayI3();
}

// I1 extends two interfaces.
// This demonstrates multiple inheritance through interfaces.
interface I1 extends I2, I3 {

    // Method declared in Interface I1
    void displayI1();
}

// C1 implements I1.
// Since I1 inherits I2 and I3, C1 must implement
// displayI1(), displayI2(), and displayI3().
class C1 implements I1 {

    public void displayI2() {
        System.out.println("Method of Interface I2");
    }

    public void displayI3() {
        System.out.println("Method of Interface I3");
    }

    public void displayI1() {
        System.out.println("Method of Interface I1");
    }

    // Method belonging to Class C1
    void displayC1() {
        System.out.println("Method of Class C1");
    }
}

public class Question {

    public static void main(String[] args) {

        // Creating an object of C1.
        // The object can access methods from I1, I2, I3,
        // and the C1 class.
        C1 obj = new C1();

        obj.displayI2();
        obj.displayI3();
        obj.displayI1();
        obj.displayC1();
    }
}
```

Expected Output

```
Method of Interface I2
Method of Interface I3
Method of Interface I1
Method of Class C1
```

Multiple Test Cases and Verification

Test Case	Methods Called	Expected Result	Status
1	displayI2()	Method of Interface I2	Pass
2	displayI3()	Method of Interface I3	Pass
3	displayI1()	Method of Interface I1	Pass
4	displayC1()	Method of Class C1	Pass

Test Case	Methods Called	Expected Result	Status
5	All four methods	All four messages printed in order	Pass

Execution Verification

The program compiles successfully when saved as **Question.java** and executed with a Java compiler. The main method creates one C1 object and invokes all four methods. The output confirms that C1 successfully receives the contracts from I1, I2, and I3 and also executes its own method.

Result

Thus, the Java program successfully demonstrates multiple inheritance using interfaces. Interface I1 inherits from both I2 and I3, and class C1 implements I1 and provides implementations for all inherited methods. The test cases produce the expected results.

Conclusion

Java does not support multiple inheritance of classes, but it supports multiple inheritance through interfaces. A single interface can extend multiple interfaces, allowing a class that implements the child interface to inherit multiple method contracts.